Multi-Tenant Academy / Institute Management SaaS

Professional Technical Documentation

1. Business overview

Purpose
Build a multi-tenant SaaS platform that lets academies, tuition centers, training institutes, and online education providers run their full operation in one system: admissions, course delivery, attendance, exams, results, fees, communication, analytics, and mobile access.

Sri Lanka problem space
Many institutes in Sri Lanka still operate with fragmented tools: WhatsApp groups, spreadsheets, paper attendance, manual timetables, bank-slip verification, and separate systems for classes, materials, and payments. At the same time, the country has strong mobile and internet reach, which makes a cloud-first education operations platform commercially realistic. Sri Lanka had 13.9 million internet users at the end of 2025, with internet penetration at 59.7%, and 30.3 million mobile connections, equivalent to 130% of population. Government schools alone account for 3,747,544 students, showing the scale of the broader education ecosystem around which private tuition and academy services operate.

How this platform solves it
The platform centralizes academic and operational workflows into one tenant-isolated system. Institutes can onboard, configure branding, create courses, enroll students, run classes, collect fees, send reminders, publish results, and track performance without platform-owner intervention. This reduces admin effort, improves student experience, and creates recurring SaaS revenue.

Target customers
- Tuition classes
- Private academies
- Professional training centers
- Language institutes
- IT / software training institutes
- Hybrid and online learning providers
- Corporate training providers

Market opportunity
The opportunity is strongest where mobile-first education operations meet fragmented institute administration. Sri Lanka's high mobile penetration, growing internet usage, and the prevalence of private learning ecosystems create a viable local launch market. A localized payment layer is also practical: PayHere supports one-time and recurring payments, including cards, wallets, and internet banking for Sri Lankan businesses.

Competitive advantages

- Sri Lanka-first localization
- Multi-tenant SaaS from day one
- Student and lecturer mobile apps
- Local payment support
- Strong automation layer
- White-label / branded tenant experience
- Scalable architecture for global rollout
- Built-in analytics and future AI layer

2. User roles

Super Admin (platform owner)

Permissions

- Manage all tenants
- Manage plans, limits, subscriptions, invoices
- View platform-wide analytics
- Control feature flags
- Manage support tickets and tenant lifecycle
- Suspend / activate tenants

System access

- Global admin portal only
- No access to tenant academic data unless explicit support impersonation is enabled and audited

Dashboard

- Active institutes
- MRR / ARR
- plan usage
- failed payments
- support queue
- system health
- API / job monitoring

Institute Admin

Permissions

- Configure institute profile, branding, branches
- Manage staff, lecturers, students, parents
- Create courses, classes, schedules, exams
- Publish materials and results
- View finance and operational reports

System access

- Tenant admin portal
- Full tenant scope

Dashboard

- Student count
- active courses
- today's classes
- fee collection
- attendance trends
- pending actions

Institute Staff

Permissions

- Admissions
- student support
- fee recording
- timetable coordination
- basic reports
- communication tasks

System access

- Limited tenant back office

Dashboard

- New enrollments
- pending fee payments
- class rosters

- document verification
- notification queue

Lecturer

Permissions

- View assigned classes
- Mark attendance
- Upload materials
- create assignments
- review submissions
- enter marks / results where allowed

System access

- Lecturer web portal + mobile app

Dashboard

- Today's classes
- student lists
- submissions to review
- exam duties
- attendance summary

Student

Permissions

- View enrolled courses
- access schedules and materials
- submit assignments
- view attendance, exam results, fee status
- receive notifications

System access

- Student portal + mobile app

Dashboard

- Upcoming classes
- pending assignments

- latest materials
- fee due alerts
- grades overview

Parent / Guardian (optional)

Permissions

- View child attendance, fees, progress
- receive notifications
- make payments where enabled

System access

- Parent portal / simplified mobile view

Dashboard

- Child performance
- fee dues
- attendance alerts
- institute announcements

3. System modules

3.1 Institute Management

Tenant creation, branch management, branding, billing profile, subscription plan, feature toggles, academic calendar, staff setup.

3.2 User & Access Management

User creation, role assignment, RBAC, invite flows, password / SSO policies, audit logs.

3.3 Student Management

Profiles, admissions, guardian links, documents, academic status, batches, ID generation.

3.4 Lecturer Management

Profiles, qualifications, assigned classes, payroll reference fields, workload visibility.

3.5 Course Management

Course catalog, modules, pricing, duration, syllabus, delivery mode, capacity.

3.6 Class & Batch Management

Intakes, batches, class groups, academic periods, rooms, online meeting links.

3.7 Timetable & Scheduling

Recurring classes, rescheduling, room conflicts, lecturer conflict detection, holiday calendars.

3.8 Attendance System

QR/manual attendance, lecturer marking, late/absent status, attendance analytics.

3.9 Assignment Management

Assignment creation, deadlines, attachments, submissions, grading, feedback.

3.10 Learning Material Management

Notes, PDFs, videos, links, downloadable assets, version control, access rules.

3.11 Exam Management

Exam types, schedules, hall allocation, mark schemes, grading rules, invigilator assignment.

3.12 Result Management

Marks entry, moderation, GPA / percentage logic, result publishing, transcripts.

3.13 Enrollment Management

Application, approval, fee confirmation, class assignment, student onboarding.

3.14 Payment & Fee Management

Invoices, installments, recurring plans, discounts, scholarships, receipts, arrears tracking.

3.15 Notification System

SMS/email/push/in-app alerts for classes, fees, results, assignments, attendance.

3.16 Reports & Analytics

Revenue, enrollments, retention, lecturer workload, performance trends, attendance heatmaps.

3.17 Support & Audit

Activity logs, impersonation audit, issue tracking, data export, compliance history.

4. Major use cases

Institute registers to platform

1. Institute signs up.
2. System creates tenant.
3. Subscription plan selected.
4. Tenant admin receives activation link.
5. Default settings, roles, and storage buckets are provisioned.

Institute admin creates courses

1. Admin creates course template.
2. Sets price, duration, lecturer, delivery mode.
3. Creates batches/classes.
4. Publishes for enrollment.

Students enroll in courses

1. Student applies or is added by staff.
2. Admin reviews eligibility.
3. Fee invoice generated.
4. After payment/approval, enrollment becomes active.
5. Student gets portal/mobile access.

Lecturers conduct classes

1. Lecturer receives class schedule.
2. Starts class.
3. Marks attendance.
4. Uploads materials.
5. Assigns homework if needed.

Attendance recorded

1. Class opens attendance session.
2. Lecturer/staff records presence.
3. System computes stats.
4. Alerts sent for absences if configured.

Assignments uploaded and submitted

1. Lecturer posts assignment.
2. Students receive reminder.
3. Students upload submission.
4. Lecturer reviews and grades.
5. Feedback published.

Exams conducted

1. Admin schedules exam.
2. Students notified.

3. Marks entered after evaluation.
4. Moderation rules applied.
5. Results published.

Students pay course fees

1. Invoice generated.
2. Payment link shared.
3. Student pays via gateway.
4. Payment webhook updates status.
5. Receipt issued automatically.

5. User journey flows

Institute onboarding flow

Landing page → plan selection → institute registration → email/OTP verification → billing setup → tenant provisioning → admin invite → basic configuration wizard → first course setup → first student import

Course creation flow

Admin login → create course → add syllabus/modules → define pricing → assign lecturer → create batch/class schedule → publish course

Student enrollment flow

Student application / staff entry → profile creation → document upload → approval → invoice generation → payment confirmation → course enrollment → welcome notification

Attendance flow

Lecturer opens class → attendance session starts → mark present/late/absent or QR scan → save → analytics update → parent/student notification if absent

Assignment submission flow

Lecturer posts assignment → student receives push/email → student uploads file or text response → deadline validation → lecturer grading → result/feedback visible

Exam and result flow

Admin creates exam → schedule + hall + batch mapping → student admission list → exam held → marks entry → moderation → result publish → transcript/report download

Fee payment flow

Invoice issued → reminder cycle starts → payment link opened → gateway payment → webhook callback → ledger entry → digital receipt → overdue follow-up if unpaid

6. Multi-tenant SaaS architecture

Tenant registration

Each new institute becomes a tenant. The provisioning service creates:

- tenant record
- default roles
- tenant config
- storage namespace
- billing profile
- usage counters
- default academic settings

Isolation model

Recommended: **shared application, shared database, tenant_id row isolation for MVP**, then evolve to **hybrid isolation** for high-value tenants.

Isolation tiers

- Tier A: Shared DB, strict row-level tenant isolation
- Tier B: Dedicated schema per tenant
- Tier C: Dedicated database for enterprise tenants

Tenant independence

Every business object carries tenant_id. Queries, caches, files, job queues, and analytics are tenant-scoped. No institute can see another institute's data.

Subscription enforcement

Per-tenant limits:

- max students
- max active courses
- max lecturers
- storage quota
- branch count
- analytics depth
- mobile branding access
- API access

Recommended tenancy safeguards

- PostgreSQL Row Level Security for sensitive tables
- tenant-aware middleware
- tenant-aware Redis keys
- per-tenant S3 prefixing
- audit logging for cross-tenant support access
- rate limiting per tenant

7. Mobile application features

Student mobile app

- Login / biometric session restore
- Course dashboard
- timetable and class reminders
- attendance history
- materials access
- assignment submission
- exam schedules
- results and grade cards
- payment status and links
- announcements
- push notifications
- profile and support

Lecturer mobile app

- Assigned class schedule
- student rosters
- attendance marking
- material upload
- assignment posting
- submission review
- exam/mark entry
- announcements

- class reschedule acknowledgement
- push notifications

8. System architecture

Frontend

React / Next.js

- SSR/CSR hybrid admin portals
- tenant-aware routing
- dashboard UI
- form-heavy workflows
- analytics views

Mobile

React Native

- shared codebase for Android and iOS
- role-based app shells for students and lecturers
- offline caching for attendance/materials

Backend

NestJS preferred

- modular architecture
- RBAC and guards
- background job support
- cleaner domain separation than a basic Express codebase

Recommended services:

- API Gateway / BFF
- Auth Service
- Tenant Service
- Academic Service
- Finance Service
- Notification Service
- Reporting Service
- File Service

- Billing & Subscription Service

Database

PostgreSQL

- relational consistency
- strong support for multi-tenant design
- indexing and reporting
- JSONB for flexible configuration

Cache / queue

Redis

- session/token blacklisting
- job queues
- timetable/material caching
- OTP and rate limiting

Storage

Amazon S3

- assignments
- materials
- receipts
- exports
- profile media

Auth

JWT + refresh tokens, optional **OAuth / Google login**

- short-lived access token
- rotating refresh token
- device/session tracking

Notifications

Firebase Cloud Messaging

- push notifications to student/lecturer apps
- paired with email/SMS provider for fallback

Payments

For a Sri Lanka-first rollout, use **PayHere as primary local gateway** because it supports one-time and recurring payments and local rails. Stripe's official global availability page lists supported countries, and Sri Lanka is not listed there as of March 2026, so Stripe is better treated as a later international expansion gateway rather than the local default. PayPal can be an optional international checkout method depending on business model.

Cloud

AWS recommended

- CloudFront + S3 for static assets
- ECS/Fargate or EKS for containers
- RDS PostgreSQL
- ElastiCache Redis
- SQS for async jobs
- SES for email
- CloudWatch for logs/metrics
- Secrets Manager / Parameter Store

High-level flow

Client apps → API gateway → auth check → tenant resolution → domain service → PostgreSQL / Redis / S3 → event bus / queue → notification workers / analytics workers

9. Database design

Core tables

- tenants
- tenant_plans
- tenant_subscriptions
- branches
- users
- roles
- user_roles
- students
- guardians
- student_guardians
- lecturers

- courses
- course_modules
- batches
- classes
- class_sessions
- rooms
- enrollments
- attendance_records
- assignments
- assignment_submissions
- exams
- exam_registrations
- results
- invoices
- invoice_items
- payments
- payment_transactions
- notifications
- notification_logs
- materials
- announcements
- audit_logs
- feature_usage_counters

Relationship highlights

- One tenant has many users, students, lecturers, courses, payments
- One course has many batches, modules, assignments, exams
- One student has many enrollments, attendance records, submissions, results, invoices
- One lecturer has many classes, assignments, materials
- One class_session belongs to a batch/class and has many attendance records
- One invoice can have many payments / transactions

- All tenant-owned tables include tenant_id

Example schema notes

- users: auth identity, email, phone, status
- students: academic profile, reference to user
- lecturers: staff profile, reference to user
- courses: title, fee plan, duration, mode
- enrollments: student-course relationship with status
- results: exam/student/grade/remarks
- payments: transaction reference, gateway, method, status

10. API design

API style

- REST for core CRUD
- webhook endpoints for payments and notifications
- versioned namespace: /api/v1
- tenant resolved by subdomain, JWT claim, or header from gateway

Sample endpoints

Tenant / onboarding

- POST /api/v1/register-institute
- POST /api/v1/tenants/{tenantId}/activate
- GET /api/v1/tenant/profile

Auth

- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout

Courses

- POST /api/v1/courses
- GET /api/v1/courses
- GET /api/v1/courses/{id}
- PATCH /api/v1/courses/{id}

Students

- POST /api/v1/students
- GET /api/v1/students
- GET /api/v1/students/{id}

Enrollment

- POST /api/v1/enrollments
- GET /api/v1/enrollments?studentId=
- PATCH /api/v1/enrollments/{id}/status

Attendance

- POST /api/v1/attendance/mark
- GET /api/v1/attendance/class/{classSessionId}
- GET /api/v1/students/{id}/attendance

Assignments

- POST /api/v1/assignments
- POST /api/v1/assignments/{id}/submit
- GET /api/v1/assignments/{id}/submissions

Exams / results

- POST /api/v1/exams
- POST /api/v1/results/publish
- GET /api/v1/students/{id}/results

Payments

- POST /api/v1/invoices
- POST /api/v1/payments/initiate
- POST /api/v1/payments/webhooks/payhere
- GET /api/v1/payments/history

Notifications

- POST /api/v1/notifications/send
- GET /api/v1/notifications

Example request

POST /api/v1/enrollments
{

```
  "studentId": "stu_123",
  "courseId": "crs_456",
  "batchId": "bat_789",
  "billingPlan": "monthly_installment"
}
```


Example response


```
{
  "success": true,
  "message": "Enrollment created",
  "data": {
    "enrollmentId": "enr_001",
    "status": "pending_payment"
  }
}
```


11. Automation features

- Automatic class reminders
- Automatic lecturer reminders before session start
- Automatic attendance absence alerts
- Automatic low-attendance warnings
- Automatic assignment due reminders
- Automatic overdue fee reminders
- Automatic receipt generation
- Automatic result publishing at scheduled time
- Automatic monthly finance reports
- Automatic subscription renewal reminders
- Automatic data backup jobs
- Automatic inactive-student re-engagement campaigns

Implementation

- Cron + queue workers
- event-driven notification service
- template engine for email/SMS/push
- retry policies and dead-letter queue

12. SaaS subscription model

Starter

For small tuition centers

- 1 branch
- up to 300 students
- up to 20 lecturers
- up to 25 active courses
- web portal
- student app
- core reports
- basic support

Professional

For growing academies

- up to 3 branches
- up to 2,000 students
- up to 100 lecturers
- advanced analytics
- lecturer app
- assignment/exam modules
- custom branding
- API/webhook access
- priority support

Enterprise

For large institutes / chains

- custom student limits
- multi-branch at scale
- SSO
- dedicated onboarding
- advanced reporting
- dedicated infrastructure option
- SLA support

- custom integrations
- white-label capability

Suggested pricing logic

Use **LKR monthly billing** for Sri Lanka launch and optional annual discount. Example positioning:

- Starter: LKR 12,000–20,000 / month
- Professional: LKR 35,000–60,000 / month
- Enterprise: custom quotation

These are product strategy estimates, not current market quotes.

13. Security

Authentication

- JWT access tokens
- refresh token rotation
- password hashing with Argon2 or bcrypt
- optional MFA for admins

Authorization

- RBAC by role + permission matrix
- tenant-scoped claims
- record-level authorization checks

Data protection

- TLS in transit
- encryption at rest
- signed URLs for private files
- secure secrets storage
- audit logging

Payment security

- never store raw card data
- gateway-hosted checkout or tokenized payment flow
- webhook signature verification
- finance event audit trail

Application security

- rate limiting
- WAF
- CSRF protection where relevant
- secure headers
- input validation
- SQL injection prevention via ORM/query builder
- SAST/DAST in CI/CD

Operational security

- environment separation
- least-privilege IAM
- backup and restore drills
- centralized logging
- anomaly alerts

14. Startup budget estimation

These are realistic **Sri Lanka startup** estimates for building an MVP and launching.

Lean MVP team for 4–6 months

- Product / founder oversight: founder-led
- 1 full-stack backend engineer
- 1 frontend web engineer
- 1 React Native engineer
- 1 UI/UX designer part-time
- 1 QA engineer part-time
- 1 DevOps/cloud engineer part-time

Estimated monthly team cost

- Backend engineer: LKR 250,000–450,000
- Frontend engineer: LKR 220,000–400,000
- React Native engineer: LKR 250,000–450,000
- UI/UX part-time: LKR 80,000–180,000
- QA part-time: LKR 80,000–150,000

- DevOps part-time: LKR 100,000–220,000

Total monthly team cost: about **LKR 980,000 to 1,850,000**

4–6 month MVP build

Estimated build cost: LKR 4 million to 11 million, depending on whether founders contribute directly and whether some roles are contract-based.

Cloud / tools monthly at early stage

- AWS / cloud hosting: LKR 40,000–180,000
- email/SMS/push tools: LKR 10,000–60,000
- monitoring/logging/tools: LKR 10,000–40,000
- domain, SSL, misc SaaS tools: LKR 5,000–20,000

Initial monthly infra: LKR 65,000 to 300,000

Marketing launch budget

- branding/landing page assets: LKR 100,000–300,000
- digital ads + sales outreach: LKR 100,000–500,000 / month
- demos/onboarding material: LKR 50,000–150,000

Recommended startup budget

For a serious first launch: **LKR 6 million to 14 million** runway target.

15. Development roadmap

Phase 1 — Discovery and architecture (2–4 weeks)

- requirements refinement
- UX flows
- multi-tenant architecture decisions
- schema design
- backlog and sprint planning

Phase 2 — MVP development (10–14 weeks)

- auth and tenancy
- institute admin portal
- course/student/lecturer modules
- class scheduling
- attendance

- materials
- basic payments
- notifications
- basic reports

Phase 3 — Beta release (4–6 weeks)

- pilot institutes
- bug fixing
- performance tuning
- onboarding flow improvements
- payment/webhook hardening
- audit and logs

Phase 4 — Mobile applications (6–10 weeks)

- student app
- lecturer app
- push notifications
- assignment submission
- attendance and result views

Phase 5 — Public launch

- billing automation
- support workflows
- marketing site
- sales onboarding
- customer success playbooks

16. Scalability plan

At 100 institutes

- single region
- shared DB with strong indexing
- Redis cache
- horizontal API scaling
- S3 storage

- nightly analytics jobs

At 1,000 institutes

- read replicas
- queue-based background processing
- partition heavy tables by tenant/date
- CDN for files
- separate reporting workloads
- better observability and autoscaling

At 10,000 institutes

- hybrid tenancy with enterprise database isolation
- microservice split for billing, notifications, analytics
- event streaming
- regional deployment strategy
- search/index service for materials and records
- data warehouse for analytics
- sharding or dedicated clusters for hot tenants

Key scaling strategies

- stateless APIs
- background job offloading
- caching
- DB indexing and partitioning
- object storage
- per-tenant throttling
- feature flag rollouts
- strong observability

17. Future AI features

- AI student performance risk scoring
- AI attendance anomaly detection
- AI recommended study materials
- AI fee default prediction

- AI batch scheduling optimization
- AI lecturer workload balancing
- AI tutor assistant for course FAQs
- AI-generated progress summaries for parents
- AI assignment similarity / plagiarism indicators
- AI enrollment forecasting for institutes
- AI chatbot for student support and admissions

Recommended MVP scope

To reduce cost and reach market faster, the first release should include:

- multi-tenant onboarding
- institute admin portal
- student and lecturer management
- course + batch + timetable
- attendance
- materials
- payments
- notifications
- student mobile app
- basic lecturer mobile features
- analytics v1

Leave advanced exam engines, AI, parent portal, payroll, and enterprise SSO for later phases.

Recommended tech stack summary

- **Web:** Next.js
- **Mobile:** React Native
- **Backend:** NestJS
- **DB:** PostgreSQL
- **Cache/Queue:** Redis + BullMQ
- **Storage:** S3
- **Infra:** AWS ECS/Fargate or EKS
- **Auth:** JWT + refresh + optional OAuth

- **Payments:** PayHere first, Stripe later for global expansion
- **Notifications:** FCM + email/SMS provider
- **Monitoring:** CloudWatch + Sentry + OpenTelemetry

This design is suitable for developers, founders, investors, and product managers because it balances local practicality, SaaS monetization, and long-term scale.